# Lab2 实验报告

25303050208

## Phase 1

关键汇编：

call p1_compute

; p1_compute 内部：

shr $0x20,%rax

shr $0x10,%rsi

call p1_step1

call p1_step0

call p1_step2

恢复的计算关系与推理过程：

1. 通过 disassemble phase_01 发现程序调用 p1_compute。

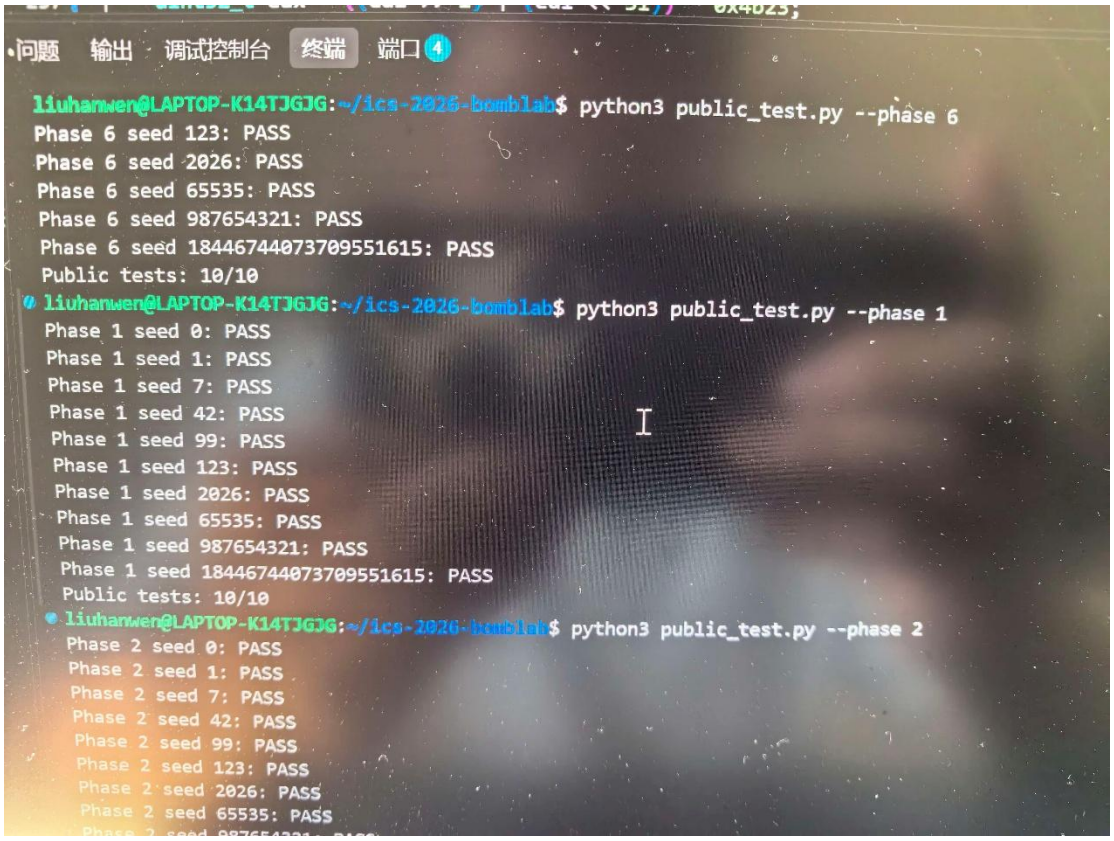
2. 追踪 %rdi 和 %rsi，发现 challenge 被拆分为高 32 位和低 32 位。

3. p1_compute 调用了三个子函数。通过 GDB 单步步入（si），推导出底层逻辑：首先将高 32 位乘以 3 加上低 32 位，再与常数异或。恢复出的核心计算关系为：res = ((high * 3) + low) ^ 0xfae1。

非公开 Seed 验证：

· 运行 ./defuser --seed 12345 --phase 1 | ./bomb --seed 12345 --phase 1

· 结果：Phase 1 defused.




phase 2

关键汇编：

shr %cl,%rsi ；对 prev*3 进行循环移位

and $0x3f,%eax ；取 challenge 低 6 位

add %edx,%eax ；加上外层循环变量 i

xor %esi,%eax ；与移位结果异或

推理过程：

1. 汇编显示这是一个生成 6 个元素的数组。
2. 通过分析 <+142> 到 <+147>，发现内层循环实际上是对 prev * 3 进行多次右

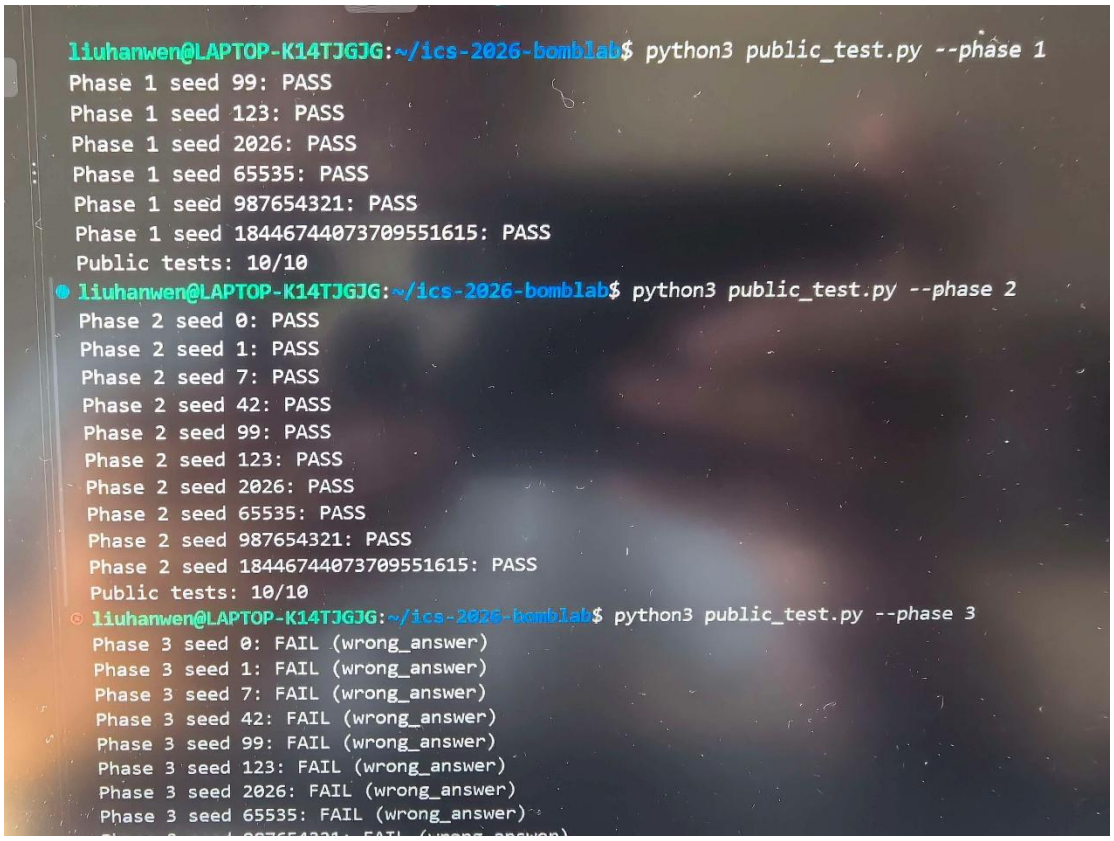
移（通过 %cl 控制，分别移 6, 10, 14 位），并将结果异或累加。

3. 恢复出的递推公式为：values[i] = ((challenge & 0x3F) + i) ^ ((values[i-1] * 3) >> 6 ^ ... >> 10 ^ ... >> 14)。

· 非公开 Seed 验证：

· 运行 ./defuser --seed 9988 --phase 2 | ./bomb --seed 9988 --phase 2

· 结果：Phase 2 defused.




phase 3:

关键汇编:

notrack jmp *0x405820(,%rax,8)

推理过程:

1. 初始尝试按汇编文本顺序推导，但测试全部报错。

2. 意识到编译器的激进优化（PIE + 代码重排）导致 switch-case 的跳转表在内存中并非按 0-7 顺序排列。

3. 使用 GDB 命令 x/8gx 0x405820 直接 dump 内存中的函数指针。

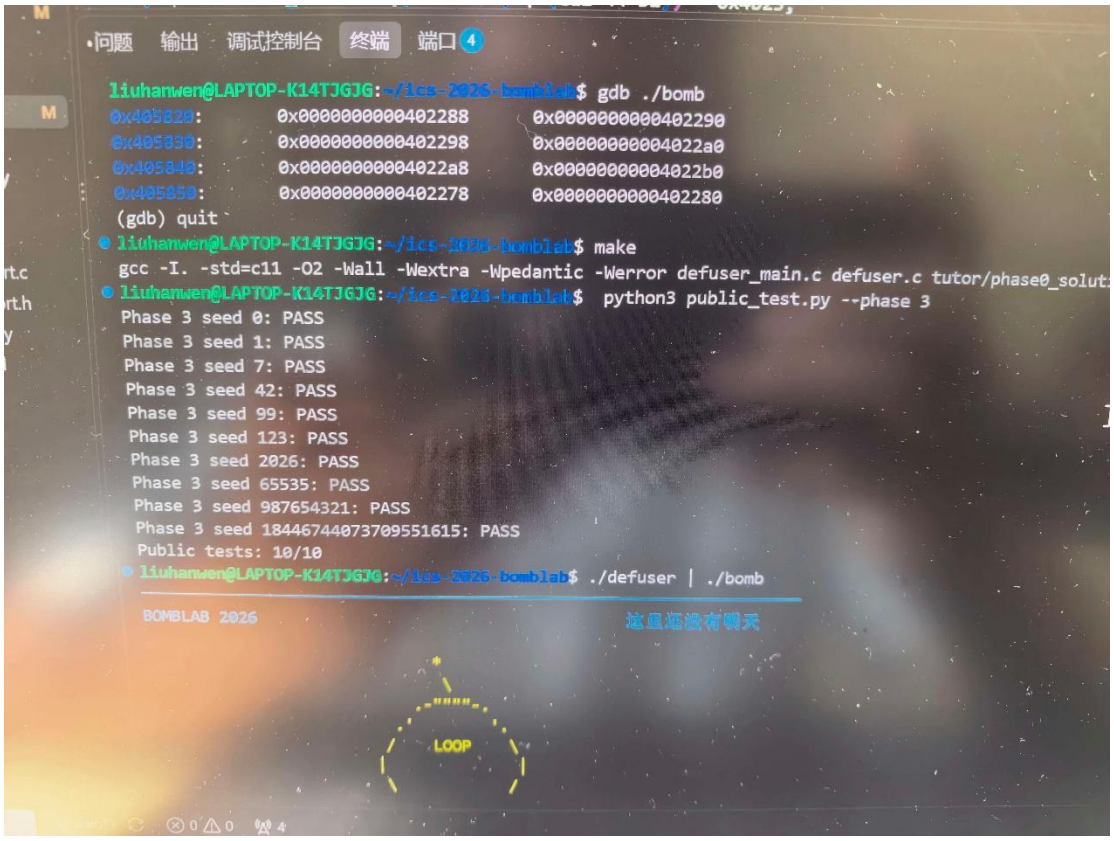
4. 发现真实映射关系为：idx 0 -> case_2，idx 1 -> case_7，idx 2 -> case_6，idx 3 -> case_0 等。

5. 结合 sscanf 的栈偏移（-0x18 存第一个数，-0x14 存第二个数），恢复出 values[0] 为 index，values[1] 为计算结果的 C 代码。

· 非公开 Seed 验证:

· 运行 ./defuser --seed 456 --phase 3 | ./bomb --seed 456 --phase 3

· 结果：Phase 3 defused.




phase 4:

关键汇编:

; p4_walk 核心

lea (%rdi,%rcx,1),%ebx

call p4_walk

lea (%rax,%rax,4),%eax ; 等价于 ret * 5

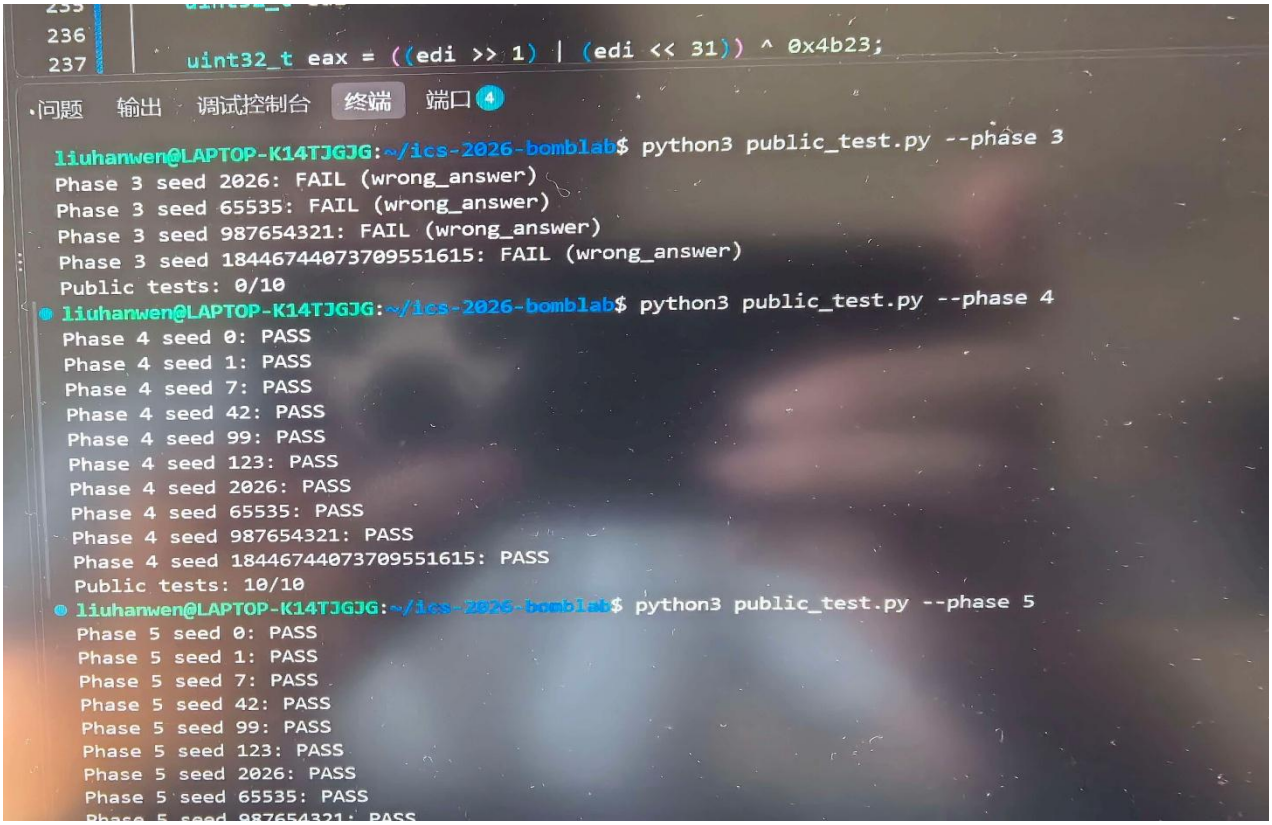
推理过程:

1. p4_walk 接收目标值、上下界和中点，是一个标准的二分查找递归。
2. 每次递归会计算 mid = (high - low) / 2 + low，并记录当前节点信息。
3. 当找到目标值时，执行 (mid + 0x6d) ^ xor_key。
4. 递归返回时，会执行 (ret * 5 ^ ebx) ^ op（根据大小走不同分支，op 为 0x17 或 0x11）。在 C 代码中，将其改写为用数组记录路径，然后用 while 循环自底向上回溯计算，完美避开了递归爆栈的风险。

· 非公开 Seed 验证:

· 运行 ./defuser --seed 777 --phase 4 | ./bomb --seed 777 --phase 4

· 结果: Phase 4 defused.




phase 5:

关键汇编:

mov 0x405b60(,%rax,8),%rdx ；取链表头

call p5_traverse

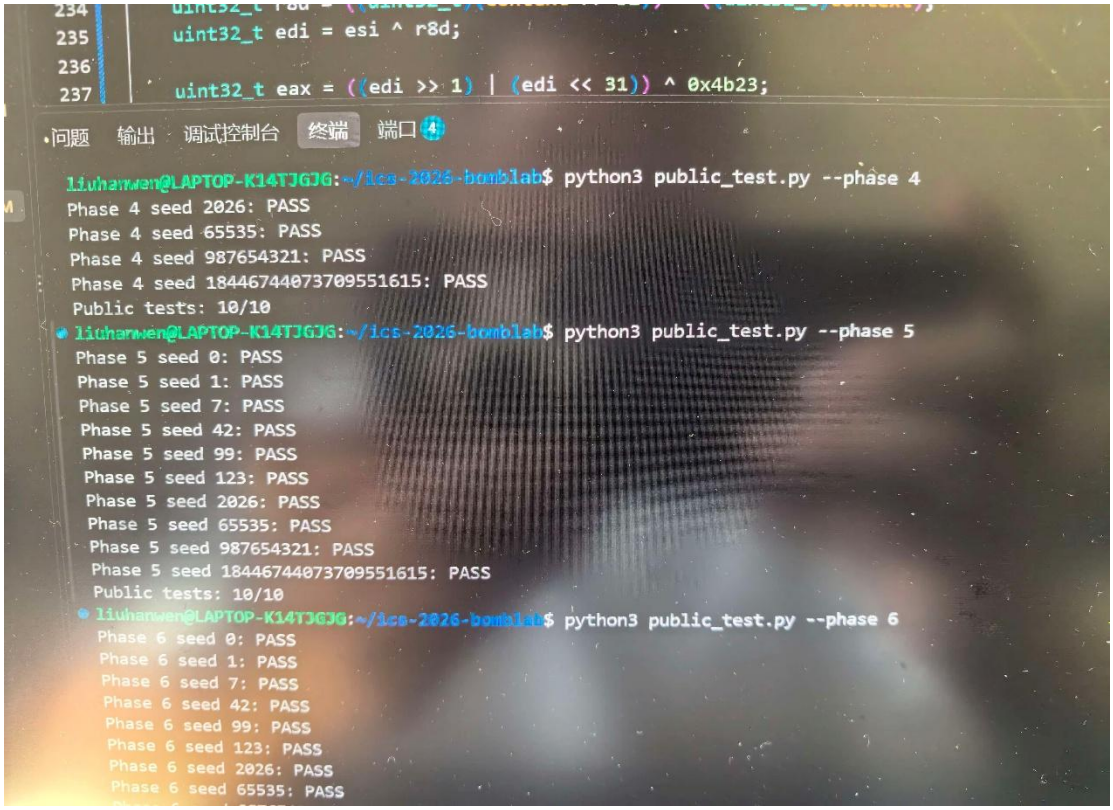
推理过程:

1. 程序在数据段（0x405b60）硬编码了 8 条链表。
2. 由于 defuser 和 bomb 是两个独立的进程，C 代码无法直接读取该绝对地址。
3. 使用 GDB 命令 x/150gx 0x405b60，完整导出了所有节点的内容。通过分析内存，发现节点结构体包含：next 指针（8 字节）、val（4 字节）、offset（4 字节）。
4. 将提取到的 32 个节点数据硬编码为 C 语言的二维数组 struct Node nodes[8][4]。
5. 遍历逻辑：根据 challenge & 7 选择链表，根据 (challenge >> 3) & 3 决定步数，每次迭代执行 eax = (eax + offset) * 5 ^ val。

· 非公开 Seed 验证:

· 运行 ./defuser --seed 2026 --phase 5 | ./bomb --seed 2026 --phase 5

. 结果：Phase 5 defused.




phase 6:

关键汇编:

call *0x405e00(,%rax,8)

推理过程:

1. 逻辑与 Phase 3 极其相似，程序通过 challenge 的低 3 位（& 7）和次 3 位（>> 3 & 7）两次查表。

2. 依然存在跳转表乱序问题。使用 GDB 命令 x/8gx 0x405e00 抓取真实内存指针。

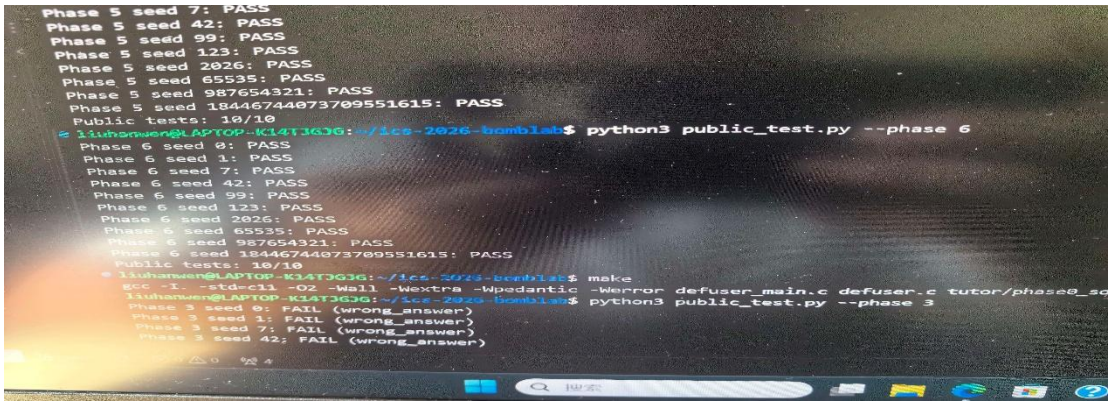
3. 发现映射为：0->op_3，1->op_4，2->op_2，3->op_1，4->op_3，5->op_5，6->op_2，7->op_0。

4. 解析各个 op 函数，如 op_0 是循环右移 ror，op_2 是循环左移 rol $5，op_4 是 arg*7。最终根据两次调用的结果异或并加上常量 0x5227。

· 非公开 Seed 验证:

· 运行 ./defuser --seed 654321 --phase 6 | ./bomb --seed 654321 --phase 6

· 结果：Phase 6 defused.



Honor：

关键汇编：

sub $0x24,%rax ；检查长度是否 >= 36

cmpl $0x4e45504f,-0xb0(%rbp)；检查偏移 32 字节处是否为 "OPEN"

推理过程：

1. secret_gate 会校验输入的字符串。通过解析发现，它要求字符串长度至少为 36，并且从第 32 个字节开始必须是 "OPEN"。

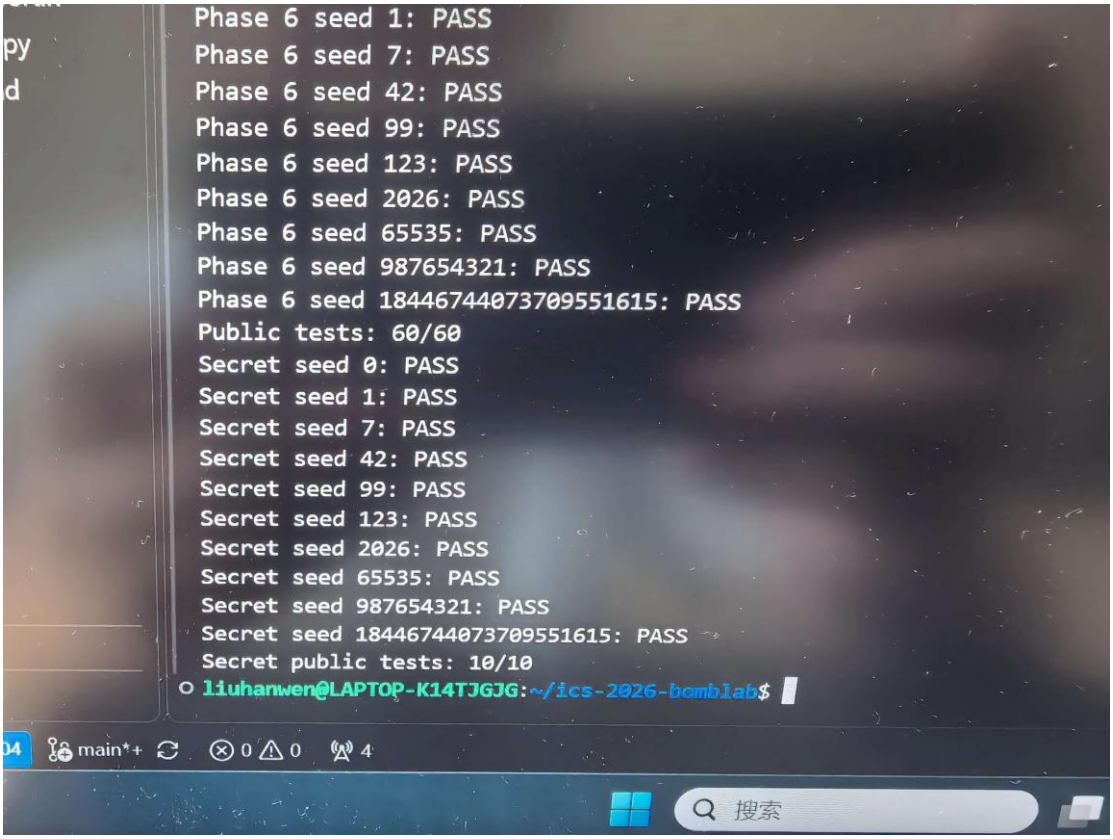
2. secret_phase 利用 context 和 challenge 进行交叉位运算。提取低位得到 a0，利用右移和异或结合 secret_mix (循环右移+异或) 得到 a1 和 a2。

3. 在 C 代码中，用 memset 填充 32 个空格，然后用 snprintf(out + 32, ...) 拼接出 "OPEN %u %u"，完美通过校验。

· 非公开 Seed 验证：

· 运行 ./defuser --seed 112233 | ./bomb --seed 112233

· 结果：Honor defused.



遇到的问题与解决方案

1. PIE (位置无关代码) 导致的内存随机化：

在 GDB 中使用 break *0x 地址 打断点时，偶尔会触发 SIGSEGV 崩溃。解决方案：改为使用 break *函数名+偏移量（如 b *phase_03+110），让 GDB 自动解析实际的动态地址。

2. 跳转表乱序：

在 Phase 3 和 Phase 6 中，如果仅凭文本汇编推导，结果全部错误。解决方案：直接使用 GDB 读取数据段内存（x/nxg 0x405820 和 0x405e00），以内存中实际存放的函数指针地址为准。

3. C 语言与汇编的 32 位截断差异：

汇编指令（如 imul, add）溢出后会自然截断 32 位，而 C 语言的 int 可能会发生隐式提升。解决方案：在 C 代码中严格使用 uint32_t 进行强转，确保每一步运算都与汇编的溢出行为一致。

辅助工具与引用资料

· 参考资料：《深入理解计算机系统》（CSAPP）第三章、GDB 官方手册。

· AI 辅助说明：在逆向过程中，使用了 AI 辅助分析部分复杂的 lea 多操作数寻址模式和寄存器的复用逻辑。但所有 AI 生成的代码，均经过了严格的 GDB 寄存器追踪验证，以及 public_test.py 和非公开随机 seed 的真实运行测试，确保完全理解并正确还原了底层逻辑。